

🔧 TALLER MIAX · AUTOENCODERS Y EMBEDDINGS

Deep Portfolios con **Autoencoders**

Replica el **S&P 500 Equal Weight** con solo **25 acciones...**
elegidas por una red neuronal que **tú** diseñas.

→ flechas del teclado para avanzar · **F** pantalla completa



EL RETO

Un tracker sparse

Construir una cartera de **solo 25 acciones** que siga al **S&P 500 Equal Weight** (ETF RSP, ~500 acciones) con el **menor tracking error** fuera de muestra.

500

acciones en el índice

25

acciones en tu cartera

min TE

gana quien mejor lo siga

$$TE = \sqrt{252} \sigma(r_{\text{track}} - r_{\text{bench}})$$

Desviación típica de la **diferencia diaria** de retornos frente a RSP, anualizada. **Menor = mejor seguimiento.**

LA VUELTA DE TUERCA

No eliges las 25 acciones... **las elige tu red**

La competición **no se gana construyendo carteras** — se gana construyendo el **mejor autoencoder**. 🧠

Entrenas un autoencoder sobre los retornos. El resto del pipeline (agrupar, elegir representantes, medir) es **idéntico para todos**. Lo único que cambias es la **red**.

CÓMO FUNCIONA

El pipeline, de principio a fin

1 Entrenas tu autoencoder **TÚ AQUÍ**
sobre los retornos del universo (train 2015-2021)

2 El encoder proyecta cada acción a un espacio latente
un embedding de pocas dimensiones por acción

3 K-means agrupa las acciones en 25 grupos
clusterización NO supervisada por distancia coseno; el medoide (activo real más central) de cada grupo → tu tracker

4 La webapp calcula el tracking error
y ordena el leaderboard en vivo

Los pasos 2-4 están cerrados e iguales para todos. **Tú solo tocas el paso 1.**

PREGUNTA 1 · CON SOLUCIÓN

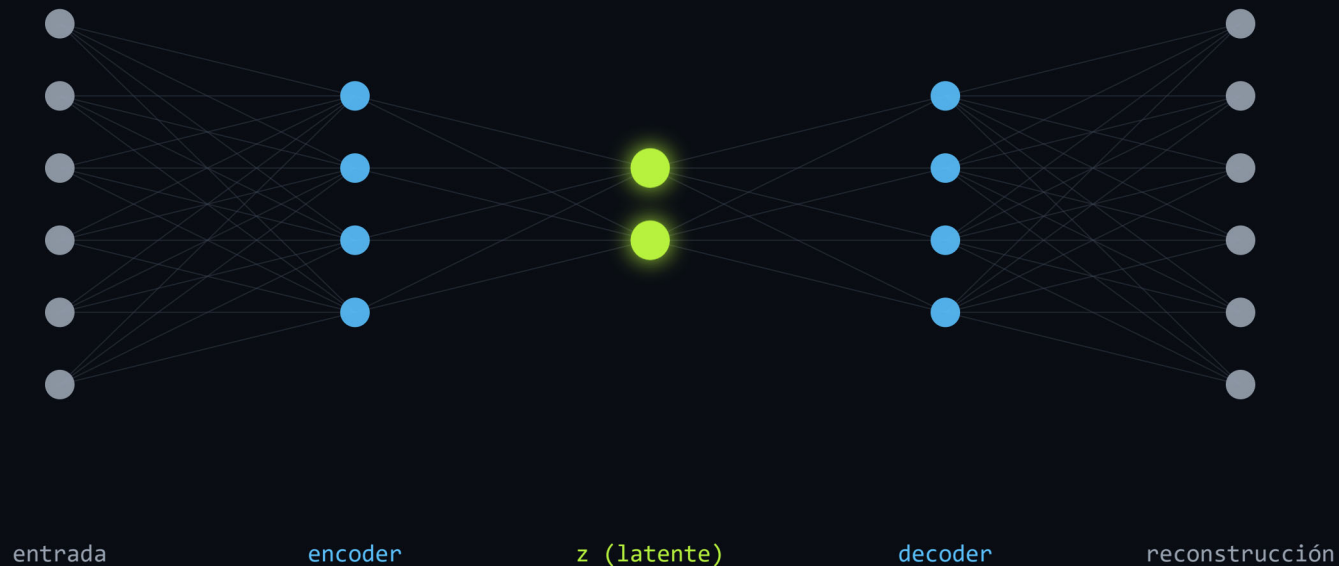
En el fondo, ¿qué aprende a hacer un autoencoder?

- ☐ A A separar los datos en clases usando etiquetas (gato, perro, fraude...)
- ☒ B A reconstruir su propia entrada obligándola a pasar por un cuello de botella
- ☐ C A predecir el número premiado de la lotería a partir de los precios 🎲
- ☐ D A ordenar los activos de más a menos rentable

Es **autosupervisado**: el objetivo es la **propia entrada** x , no una etiqueta externa. El cuello de botella lo obliga a **comprimir** en vez de copiar.

Encoder → latente (z) → decoder

Comprime la entrada por un **cuello de botella** (z) y la **reconstruye**: al no poder copiarla, se ve forzado a quedarse con los **factores comunes**.



Ojo — no confundir: la **dimensión latente** (factores que describen cada acción) y el **nº de acciones del tracker** (fijo en 25) son cosas distintas. Puedes usar 3 factores y, sobre esa nube, elegir 25 representantes.

PREGUNTA 2 · CON SOLUCIÓN

¿Qué relación hay entre el PCA y un autoencoder?

☒ El PCA es, en esencia, un autoencoder lineal; con no-linealidad el AE ajusta curvas que el PCA no capta

☐ B No tienen nada que ver: uno es estadística y el otro una red neuronal

☐ C El PCA reconstruye siempre mejor que cualquier autoencoder

☐ D El PCA es un autoencoder con infinitas capas y mucho dropout 🤖

Con encoder/decoder lineales y MSE, el AE recupera el **mismo subespacio que el PCA**. La ventaja del AE es la **no-linealidad**: sigue estructuras curvas. Por eso el PCA es el **baseline** del taller.

 LA VARA DE MEDIR

PCA = un autoencoder **lineal**

PCA busca las direcciones de máxima varianza: es un autoencoder **sin no-linealidades**. Está en el leaderboard como el **competidor fantasma** 🐻.

La pregunta del taller: **¿puede tu red no lineal batir al método lineal clásico?** Tu AE solo aporta valor si su tracking error **baja del de PCA**.

PREGUNTA 3 · CON SOLUCIÓN

Pones un espacio latente ENORME (igual o mayor que la entrada). ¿Qué riesgo hay?

- ☐ A Ninguno: cuanto más grande el latente, mejor el resultado siempre
- ☐ B Que la red se aburra y empiece a inventarse los datos 🤪
- ☒ C Que aprenda la función identidad: copie la entrada sin extraer estructura
- ☐ D Que el entrenamiento tarde más, pero el resultado sea idéntico

Con capacidad excesiva el AE puede aprender la **identidad** (copiar la entrada). Un error de reconstrucción bajo en train **no garantiza** buena representación → sobreajuste, y en el taller puede que **ni bata a los monos**.

 ELEGIR EL TAMAÑO DEL LATENTE

Ni tan pocos, ni demasiados

LATENTE MUY PEQUEÑO

Pierde estructura: el error de reconstrucción sube, no captura los factores comunes.

LATENTE DEMASIADO GRANDE

Casi no comprime: puede aprender la **identidad** y **sobreajustar**. Baja el error en train sin mejorar fuera de muestra.

Es un **hiperparámetro**: se prueba y se mira el error en **validación**. El "codo" señala el punto de rendimientos decrecientes.

PREGUNTA 4 · CON SOLUCIÓN

En el taller normalizamos los retornos (z-score) ANTES de entrenar el AE. ¿Por qué?

- ☐ A Para que el entrenamiento tarde menos
- ☐ B Porque despista a los demás equipos 🕵️
- ☒ C Para que la red no dé un peso excesivo a los días de mayor escala (volatilidad)
- ☐ D Porque Keras da un error si no lo haces

Sin normalizar, la red da **peso excesivo** a los días de mayor escala (los más volátiles). **Regla de oro:** μ y σ se estiman **solo con TRAIN** y **jamás** con validación/test — usar el z-score de valid/test es **fuga de información** (look-ahead) y falsea la generalización. En este taller, además, valid/test **ni siquiera pasan por el AE**.

Una fila por acción, **z-score** por día

Partimos de **retornos simples diarios** (precios ajustados). La matriz tiene una **fila por acción** y sus días de train como columnas. El encoder proyecta cada acción al latente.

$$\tilde{r}_{i,t} = \frac{r_{i,t} - \mu_t}{\sigma_t}$$

Estandarizamos **cada día** (sección transversal) con μ y σ estimadas **solo en TRAIN**. **Nunca** se calcula la normalización con validación/test (sería **fuga de información**); de hecho, aquí valid/test **ni pasan por el AE**. Así, además, ningún día muy volátil domina el aprendizaje.

La matriz que entra al autoencoder tiene una fila por acción y sus días de train como columnas. ¿Cuántas filas (acciones) tiene?

- ☐ A 25 — las que acaba teniendo tu tracker
- ☒ **✓** **≈342** — todas las acciones del universo (las presentes en el índice durante toda la ventana)
- ☐ C ≈500 — todas las del S&P 500 Equal Weight
- ☐ D ≈1762 — una por cada día de train 📅

El AE se entrena con **todo el universo** (~342 acciones): una **fila** por acción y ~1762 **columnas** (días de train) → matriz ~342×1762. Las **25** son el resultado del clustering (la *salida*), no la entrada.

La posición **guarda el significado**

El latente ya **es un embedding**: un vector denso donde **lo parecido queda cerca**. Dos acciones que co-mueven acaban próximas — y se agrupan por sector **sin supervisión**.

¿Cómo medimos el parecido? Con la **similitud coseno**: mira la **dirección** del vector (cómo co-mueven), no su longitud. Es la distancia natural de los embeddings — y la que usa la selección del taller.

PREGUNTA 6 · CON SOLUCIÓN

¿Cuál de estas afirmaciones sobre el espacio de embeddings es CORRECTA?

- ☐ A Sumar los embeddings de dos empresas te da su fusión real en bolsa
- ☐ B La longitud del vector indica cómo de rica es la empresa
- ☐ C El embedding predice el precio exacto de la acción de mañana
- ☒ **Restando y sumando vectores emergen analogías: rey – hombre + mujer ≈ reina 👑**

En embeddings bien entrenados **emergen direcciones con significado** (género, realeza...). No es regla exacta, sino una regularidad — el ejemplo clásico de **word2vec**. Ojo: esta vez la opción "graciosa" era la **verdadera** 😊.

PREGUNTA 7 · CON SOLUCIÓN

Tras agrupar los activos por su embedding, ¿qué es el 'medoide' de cada grupo?

- ☒ El activo real más central del grupo, que hace de representante
- ☐ B El promedio (media) de todos los vectores, aunque no exista como activo real
- ☐ C El activo con mayor capitalización del grupo
- ☐ D El que está más lejos de todos, para dar variedad 🤪

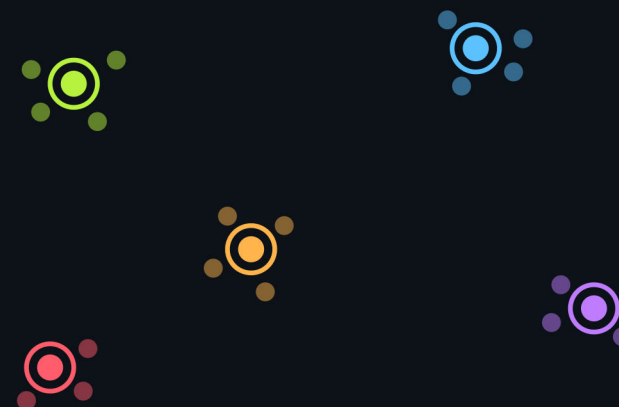
El medoide es el punto **más central** y **ES un activo real** (no un promedio). Un representante por grupo → los **25 activos** del tracker cubren todo el universo sin redundancia.

Del latente a los 25 representantes

1 Cada acción → un punto
el encoder comprime su serie de retornos en un vector latente (su embedding).

2 K-means agrupa en 25 grupos
por similitud coseno (spherical k-means): las que co-mueven caen juntas.

3 Un medoide por grupo
el activo real más central → 25 representantes. Peso = tamaño del grupo.



~342 acciones en el latente · 25 grupos (K-means) · ○ = medoide

Un representante por grupo → los **25 activos** cubren todo el universo **sin redundancia**. Este paso es **fijo e igual para todos**: tú solo cambias el autoencoder que produce el latente.

② DERIVACIÓN DEL TRACKER (FIJA)

25 grupos → 25 medoides

Se agrupan las acciones por **similitud coseno** del latente en 25 grupos. Cada representante es el **medoide**: el activo real más central del grupo. Los pesos van por tamaño de grupo.

Idéntico en el notebook y en el servidor. Si lo que envías no es coherente con tu latente, la webapp lo **rechaza**.

🔧 NUESTRO UNIVERSO VS RSP

Casi idénticos... salvo el retorno

Equal-Weight de nuestro universo (~342 acciones) frente a RSP. Medido en servidor (incluye el test que no veis).

Ventana	TE	Corr.	CAGR univ	CAGR RSP	Vol univ	Vol RSP	MaxDD univ	MaxDD RSP
Train 15-21	1.65%	0.996	14.9%	12.6%	19.0%	19.0%	-38.8%	-39.0%
Valid 22-23	2.16%	0.994	2.9%	0.2%	18.5%	19.0%	-20.2%	-21.4%
Test 24-25	1.82%	0.992	15.5%	12.0%	14.4%	14.6%	-17.0%	-17.8%

CASI IDÉNTICO

Correlación ~0.99, volatilidad, max drawdown y tracking error (~2%): prácticamente iguales.

UNA DIFERENCIA

El CAGR de nuestro universo va ~+3 pp por encima del de RSP.

Mismo riesgo, mismo seguimiento... pero más retorno. ¿Por qué? 🤔

El CAGR de nuestro universo va ~3 pp por encima del de RSP. ¿Cuál es la razón PRINCIPAL?



Sesgo de supervivencia: el universo solo incluye las acciones que siguieron en el índice toda la ventana (las ganadoras)

B

Que nuestro universo (~342) está incompleto frente a las ~500 acciones de RSP

C

La comisión del ETF: RSP paga ~0.20 %/año y nuestro universo teórico no la paga

D

Nuestras acciones rinden más porque saben que las estamos mirando 🐼

Domina el **sesgo de supervivencia**: solo vemos a las que **sobrevivieron** en el índice. El universo incompleto y la comisión influyen, pero poco. Ojo: esto mueve el **nivel de retorno**, no la **vol**, el **maxDD** ni la **correlación** (~0.99) → no falsea la competición, que puntúa por **TE**.

SESgo DE SUPERVIVENCIA

Solo vemos a las que sobrevivieron

El universo son las acciones presentes en el índice **durante toda** la ventana. Las que cayeron o salieron no están → un panel de **ganadoras**.

SÍ LO MUEVE

El **nivel de retorno**: el CAGR del universo va $\sim +3$ pp por encima de RSP.

NO LO MUEVE

La **vol**, el **max drawdown** y la **correlación** (~ 0.99) son casi idénticos.

Puntuamos por **tracking error**, no por retorno → el sesgo es el mismo suelo para todos y no falsea la competición.

LA MÉTRICA DE LA COMPETICIÓN

Tracking Error **anualizado**

$$TE = \sqrt{252} \sigma(r_{\text{track}} - r_{\text{bench}})$$

Desviación típica de la **diferencia diaria** de retornos frente a RSP, escalada a un año.

Menor = mejor seguimiento.

Puede haber **correlación alta y TE no nulo**: se mueven en la misma dirección pero difieren en *magnitud*. El TE mide justo esa diferencia.

PREGUNTA 9 · CON SOLUCIÓN

El tracking error del taller (menor = mejor), ¿qué mide en realidad?

- ☐ A La rentabilidad total de tu cartera
- ☐ B El nº de veces que has enviado al leaderboard 🚩
- ☒ C La calidad de tu espacio latente: mejor latente → representantes menos redundantes → menor TE
- ☐ D Cuántas acciones tecnológicas has metido en la cartera

El score mide **directamente** la calidad del latente y sus embeddings. Un mejor espacio da representantes que **cubren mejor** el universo → replican mejor el índice → **menor** tracking error.

 EL SUELO IRREDUCIBLE

El TE **nunca llega a 0**

Y no será culpa de tu autoencoder: hay un **error irreducible, idéntico para todos**. Tu trabajo es **acercarte a ese suelo** mejor que nadie.

~2%

suelo · universo completo (~342)

~4.8%

PCA · el competidor fantasma

¿?

tu AE · ¿cuánto te acercas?

¿DE DÓNDE SALE EL SUELO DE ~2 %?

Equal-Weight del universo **vs RSP**

≈342 acciones, rebalanceo trimestral como RSP. Medido en servidor (incluye el test que no veis).

Ventana	TE	Corr.	CAGR univ	CAGR RSP	Vol univ	Vol RSP	MaxDD univ	MaxDD RSP
Train 15-21	1.65%	0.996	14.9%	12.6%	19.0%	19.0%	-38.8%	-39.0%
Valid 22-23	2.16%	0.994	2.9%	0.2%	18.5%	19.0%	-20.2%	-21.4%
Test 24-25	1.82%	0.992	15.5%	12.0%	14.4%	14.6%	-17.0%	-17.8%

Composición (≈342 vs ≈500) — causa dominante: faltan ~160 nombres; el co-movimiento diario difiere.

Rebalanceo — replicamos el trimestral de RSP → esta fuente casi desaparece (solo residuo por fechas).

Mecánica del ETF — costes de sus rebalanceos, cash drag, muestreo, timing de dividendos.

Comisión (~0.20 %/año) — arrastre casi constante; mueve el nivel (CAGR), poco el TE.

A esto tu tracker **añade su propio TE**: usa solo **25 acciones** (no 342/500) y depende de la **calidad de tu autoencoder**. Ahí es donde se compite.

Validación 2022–2023: otro régimen

Retorno anual de RSP (total return). Casi todo verde... salvo 2015, 2018 y sobre todo 2022.

TRAIN 2015–2021	2015 -2.7%	2016 +14.5%	2017 +18.5%	2018 -7.8%	2019 +28.9%	2020 +12.7%	2021 +29.4%
VALID. 2022–2023	2022 -11.6%	2023 +13.7%					
TEST 2024–2025	2024 +12.8%	2025 +11.2%					

CAGR por periodo: **train +12.6 %** · **validación +0.2 %** · **test +12.0 %**. Validación quedó **casi plana** → otro régimen, y por eso una prueba real de **generalización**: el AE debe captar la **estructura transversal** (qué co-mueve), no el nivel.

❄ LAS REGLAS DEL JUEGO

Validación libre • **test una sola vez**

2022 - 2023

Validación

Iteras **sin límite**. El leaderboard en vivo se basa en ella. Tu campo de pruebas.

2024 - 2025

Test — el juez real

Una **única evaluación final**, modelo congelado, irreversible. **El test nunca sale del servidor.**

Reenviar sin parar mirando validación **es sobreajustar a validación** — el error que este máster enseña a evitar.

PREGUNTA 10 · CON SOLUCIÓN

¿Por qué el test (2024–2025) se guarda y se consulta UNA sola vez?

- ☐ A Porque el servidor es lento y no aguanta más consultas
- ☒ B Porque cada decisión tomada mirándolo lo contamina como estimador del error de generalización
- ☐ C Porque el test tiene muchos menos días que la validación
- ☐ D Porque en test los precios están en otra moneda 🔄

Validación sirve para **elegir hiperparámetros**; el test **estima la generalización** y por eso se reserva. Reenviar sin parar mirando validación = **sobreajustar a validación**, el error que este máster enseña a evitar.

🏆 CÓMO VAMOS A PUNTUAR LA COMPETICIÓN

Tu nota, **punto a punto**

 **Entrega correcta**
un envío final válido y coherente **50 puntos**

 **Bates a PCA**
tu tracking error de test baja del competidor fantasma **70 puntos**

 **Percentil vs los monos (test)**
cuanto mejor que el aleatorio sensato, más puntos **+20 puntos**

 **Podio en test**
1º → +10 · 2º y 3º → +5 **+10 puntos**

Tope 100. El máximo = 70 (bates a PCA) + 20 (bates a los monos) + 10 (1º) = **100 puntos**. Y hay un **bonus del profe** por participación en clase (según el ranking): 🏆 +25 puntos · 🥈 +20 puntos · 🥉 +15 puntos · 4º-10º +10 puntos — pero la nota final **nunca pasa de 100**.

¿Bates a un aleatorio **sensato**?

Los "monos" no son ingenuos: reparten 25 acciones **por sectores** (como el índice) y eligen al azar dentro de cada uno. Tu **percentil frente a los monos** en test vale hasta **+20 puntos** de tu nota.

+20

bates al mejor mono

+10

quedas en la mediana

+0

peor que todos

Cómo se reparten: puntos = $20 \times (\text{percentil} / 100)$, siendo el percentil el % de monos que superas. **Ejemplo** 🐒: si tu tracker traquea mejor que el 80 % de los monos $\rightarrow 20 \times 0.80 = \mathbf{+16 \text{ puntos}}$. (Percentil 95 $\rightarrow +19$ · mediana (50) $\rightarrow +10$ · último $\rightarrow +0$.)



Que gane el **mejor autoencoder**

Descarga tu material, abre el notebook, diseña tu red y sube al leaderboard. El resto del pipeline ya está resuelto — **la red es tuya.**

Diseña

Entrena

Valida

Congela y gana 🏆